

Operationalizing Document AI: A Microservice Architecture for OCR and LLM Pipelines in Production

Yao Fehlis, Benjamin Bengfort, Zhangzhang Si, Vahid Eyorokon, Prema Roman, Patrick Deziel, Devon Slonaker, Steve Veldman, Ben Johnson, Joyce Rigelo, Michael Wharton, Steve Kramer

KUNGFU.AI

Abstract

Academic research tends to focus on new models for document understanding creating a wide gap in the literature between model definition and running models at production scale. To close that gap, we present a microservice architecture that encapsulates pipelines of multiple models for classification, optical character recognition (OCR), and large language model structured field extraction as well as our experience running this pipeline on thousands of multi-page documents per hour. We describe our primary design decisions, including a hybrid classification, separation of GPU-bound inference from CPU-bound orchestration, use of asynchronous processing for the many IO-bound operations in the pipeline, and an independent, horizontal scaling strategy. Using batch profiling, we identified two surprising qualitative findings that shape production deployments: OCR, not language-model parsing, dominates end-to-end latency, and the system saturates at a concurrency determined by shared GPU-inference capacity rather than worker count. Our goal is to provide practitioners with concrete architectural patterns for building document understanding systems that work beyond the benchmark; effectively operationalizing models in production.

1 Introduction

The document understanding research community produces a steady stream of model innovations—LayoutLM [Xu et al., 2020], DocTR [Mindee, 2021], Donut [Kim et al., 2022], Pix2Struct [Lee et al., 2023], and dozens of vision-language models (VLMs)—each advancing accuracy on benchmarks like DocVQA [Mathew et al., 2021] and FUNSD [Jaume et al., 2019]. Yet a practitioner attempting to deploy these models into a production system that processes thousands of forms per day finds little guidance on the engineering required to make them work reliably.

The gap between a model checkpoint and on-demand model usage is substantial. Models must be containerized and served behind inference APIs. Documents arrive in heterogeneous formats—multi-page TIFFs, scanned PDFs, photographs—and must be normalized before processing often using GPU-accelerated operations. Classifi-

cation must route each document to the correct extraction pipeline. OCR must handle scanning artifacts, skewed pages, and degraded image quality. LLMs must be used to extract fields that must be validated and structured correctly. All of these operations must effectively wrap multiple model compute paradigms that are generally implemented efficiently for batch processing (e.g. training and multi-instance inference) but not for dynamic switching between model types while balancing CPU-, GPU-, and IO-bound operations with fixed memory resources.

Solving on-demand model usage is only the first step: moving from model usage to a production system requires addressing a wide range of other challenges from managing the differences between failures and errors to determining the configuration that best suits both the model and the workload. Other considerations such as implementing timeouts and retries to handle stochastic failures, outputting model meta information such as confidence scores, and model operation tracing for explainability blur the line between scientific research and software engineering. For deployment scenarios that involve sensitive, private, and/or proprietary information, the system architecture must support processing entirely within a secure cloud enclave and/or with an on-premise computing platform with sufficient scanning, version control, logging, and access controls. All of this must happen at scale, with observable failures, predictable latency, and, of course, *manageable cost*.

This paper describes an architecture for scalable and fault-tolerant production system for structured form processing that addresses these challenges. We also describe a system that we built using this architecture, which processes scanned, multi-page documents through a pipeline of classification, OCR, text stitching, and language-model-based field extraction. Finally we describe our experiences deploying the architecture as three microservices on Kubernetes with message queue-based work distribution and object storage for documents and how we reduced the cost of processing documents from \$0.01 per page to \$0.001 per page while maintaining a 96% accuracy.

We make three contributions. First, we describe the architecture and the design decisions behind it, including why we decompose the system into three services rather than a monolith, and how we separate GPU-bound infer-

ence from CPU-bound orchestration (§3). Second, we walk through the pipeline design in detail, including a hybrid classification strategy that balances cost and accuracy, and the flow from raw images to structured JSON output (§4). Third, we report qualitative findings from batch profiling that reveal the system’s scaling behavior and bottlenecks, together with lessons learned from production operation (§5).

2 Related Work

Document understanding models. The model landscape spans traditional OCR (Tesseract [Smith, 2007], PaddleOCR [Du et al., 2020]), layout-aware transformers (LayoutLM [Xu et al., 2020], LayoutLMv3 [Huang et al., 2022]), end-to-end image-to-text models (Donut [Kim et al., 2022], Nougat [Blecher et al., 2023]), general-purpose VLMs [Anthropic, 2024, Google DeepMind, 2024, Bai et al., 2023], hybrid OCR-into-VLM designs [Nacson et al., 2024], and CPU-optimized open models such as Docling [Auer et al., 2024]. Each advances accuracy on benchmarks but leaves open how to compose these components into a production system.

Production ML systems. System-level descriptions of production ML deployments remain less common than model papers. TFX [Baylor et al., 2017] describes Google’s end-to-end ML platform. Uber’s Michelangelo [Hermann and Del Balso, 2017] and Facebook’s FBLearner [Dunn, 2016] address ML workflow management. For document processing specifically, enterprise platforms like ABBYY, Kofax, and cloud services (AWS Textract, Google Document AI, Azure Form Recognizer) exist as commercial offerings, but their architectures are proprietary and undocumented in the literature. Recent work has begun to describe enterprise and multimodal document-processing systems: IDP Accelerator [Islam et al., 2026] presents an agentic document-intelligence framework with document splitting, extraction, analytics, and compliance validation; MMORE [Sallinen et al., 2025] describes a modular distributed pipeline for multimodal retrieval-augmented generation and extraction across heterogeneous file types; and domain-specific systems combine OCR, classifiers, and VLMs for claims or copy-heavy enterprise extraction [Cheng et al., 2026, Wang and Shen, 2025]. These systems show that large-scale document AI is an active area; our focus is the service-level architecture and operational lessons for a structured-form extraction pipeline.

OCR versus multimodal extraction. OCR-free and image-native document models such as Donut [Kim et al., 2022] motivate simpler pipelines that send page images directly to a VLM. Recent benchmarking suggests that powerful multimodal LLMs may match OCR-enhanced approaches for some business-document extraction tasks [Shen et al., 2026]. Our system uses OCR-first

extraction for cost control, auditability, page-level intermediate artifacts, and compatibility with text-only parsing models, but the architecture treats this as a configurable tradeoff rather than a permanent modeling assumption. We refer readers to our prior practical guide [Fehlis et al., 2025] for a capability-dimension framework (text recognition, structural understanding, output flexibility, spatial awareness, task adaptability) that can inform this OCR-versus-VLM choice on a per-deployment basis.

Document retrieval. ColPali [Faysse et al., 2024] introduces late-interaction embeddings for document retrieval, enabling efficient page-level search without OCR. This retrieval-augmented paradigm complements extraction pipelines like ours by enabling selective processing of relevant pages from large document collections.

3 System Architecture

The architecture decomposes document understanding into three microservices, each independently deployable and scalable (Figure 1). This decomposition reflects a fundamental insight: the computational profiles of inference (GPU-bound, high memory) and orchestration (CPU-bound, I/O-heavy) differ enough that coupling them wastes resources and limits scaling flexibility.

By decoupling inference, CPU-bound services primarily utilize I/O-heavy operations required for document processing, allowing them to take advantage of asynchronous coroutines without the need for internal parallelization. Although this approach means that inference services cannot be scaled using batch-processing without buffering, it does allow for independent scaling of each service. This architecture is therefore well-suited to tuning to specific workloads without over-provisioning resources that would otherwise sit idle during I/O operations.

3.1 Gateway: Ingestion Service

The Gateway service is the system’s entry and exit point. It accepts document submissions through two inbound paths: a REST API for synchronous submissions from clients and an ingestion queue for asynchronous handoff from upstream pipelines. The Gateway is responsible for storing page images in object storage, creating tracking records in a relational database, and enqueueing document IDs to notify workers the document is ready for processing. It also serves a web-based inspection UI for human review of extraction results and reports processing status via a status queue.

The Gateway is almost entirely I/O-bound and must scale to the expected rate of ingestion throughput. For example, commercial scanners are capable of scanning 150 pages per minute [Ricoh, 2024] at 300 dpi producing page images of 2-90 MiB requiring a bandwidth averaging 225 MiB/s. For physical document scanning workloads, this

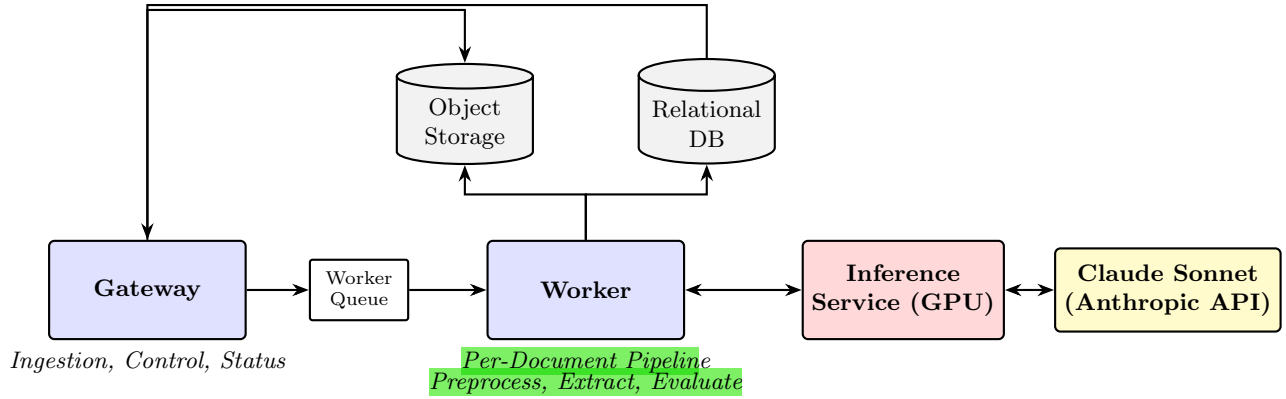


Figure 1: System architecture. The Gateway accepts submissions, persists page images in object storage and tracking records in the relational DB, and enqueues document IDs onto a message queue. Workers pull documents, perform CPU-bound orchestration, and call the Inference Service for GPU-bound OCR and the Anthropic API (Claude Sonnet) for VLM-based steps and language-model parsing. Separating CPU-bound orchestration from GPU-bound inference enables each tier to scale independently.

bandwidth is "bursty" as scanning throughput is limited by the size of the document feeder and normal work hours for the facility. In this simple example, it is easy to see that ingestion throughput is often fundamentally different than the throughput of the rest of the system, requiring a different scaling strategy from other document processing workloads.

Because of the difference in scaling strategy, the Gateway is intentionally lightweight—it performs no inference or extraction, nor any other CPU intensive operations. This keeps its resource footprint small (tuned to ingestion) and its failure modes simple: if the Gateway is down, no new documents enter the system, but in-flight processing continues unaffected.

3.2 Worker: Pipeline Orchestration

Workers serve as the pipeline’s execution engine. They orchestrate the entire workflow and run individual steps as dictated by the system configuration. Each worker pod runs multiple concurrent tasks (limited to a maximum number to prevent resource contention), pulling document IDs from the message queue, and executing the configured extraction pipeline. Workers download page images from object storage on demand (lazy loading), invoke the Inference Service for inference-heavy steps, and upload structured JSON results upon completion.

Workers are CPU-bound and I/O-heavy: they spend most of their time waiting for inference responses, database reads and writes, downloading and uploading images, and marshaling data between pipeline steps. We therefore use asynchronous task execution inside each worker process so one task can make progress while another waits on inference or I/O. This provides vertical concurrency within a pod, while Kubernetes provides horizontal scaling by adding more worker pods. Throughput increases with worker concurrency until the Inference Service or

downstream APIs saturate.

The effective concurrency of the system is pods $\times$ tasks per pod. With the default configuration of 5 tasks per pod, a deployment of 5 worker pods provides 25 concurrent document processing slots.

3.3 Inference Service

The Inference Service isolates GPU-bound inference behind a REST API, exposing OCR models (e.g., DocTR [Mindée, 2021]) and VLM capabilities (via cloud API proxying to managed services) to workers. This separation provides three benefits:

1. **Independent scaling.** GPU nodes are expensive. Decoupling inference from orchestration means we provision GPU capacity based on inference demand, not worker count.
2. **Model swapping.** New OCR models can be deployed to the Inference Service without touching worker code. For example, we have swapped among DocTR, Docling [Auer et al., 2024], and SmolDocling with configuration changes only.
3. **Resource isolation.** DocTR requires ~ 800 MB of GPU memory. Running it inside worker pods would either waste GPU resources (because most worker time is non-inference) or force CPU-only inference (3–5 $\times$ slower).

An important consideration for the deployment of inference service(s) is ensuring computational *staggering*. Processing a single document is a sequential process where each step follows from the previous step. However, document processing steps can be executed in parallel—GPU operations can be performed during CPU operations, and multi-core CPUs can effectively handle multiple steps. Staggering multiple sequential processes allows this parallelism and to saturate the compute resources available if the steps are carefully designed to provide this advantage.

To achieve staggering, the inference service must use hybrid capabilities. At first, we were concerned primarily with VLM throughput and batching. However, by utilizing different models for classification, OCR, data extraction, validation, and more, we were able to achieve staggering which led to better resource utilization (and better performance for cost) than batching alone.

3.4 Queue-Driven Communication

Asynchronous job coordination flows through message queues, providing backpressure, retry semantics, and decoupled scaling. Inference steps use direct request-response service APIs; queues coordinate document ownership, status propagation, and retry behavior. Three queues serve distinct roles:

- **Ingestion queue:** External systems submit document IDs for processing.
- **Worker queue:** The Gateway enqueues validated documents; workers consume them.
- **Status queue:** Workers publish completion notifications; the Gateway consumes them for status reporting.

Queue-based communication means any service can be restarted or scaled without affecting others. If workers fall behind, messages accumulate in the worker queue, providing natural backpressure rather than cascading failures. Queues also coordinate the document processing steps, ensuring that only one worker processes a document at a time, ensuring that the state of the document is modified in an idempotent manner during processing even when a failure occurs. This guarantee allows us to use checkpointing and retry mechanisms that do not require reprocessing the entire document from the beginning.

3.5 Failure Isolation and Service Contracts

The microservice boundary is useful not only for scaling but also for failure isolation. The Gateway, Workers, and Inference Service communicate through narrow contracts: object storage is the source of truth for page artifacts, queue messages carry lightweight work references rather than full payloads, and the Inference Service exposes stable request-response interfaces for OCR and model-backed extraction. This approach keeps orchestration logic independent of model implementation details and prevents transient model-serving failures from propagating arbitrary state back into the rest of the system.

These contracts also define restart behavior. If the Gateway becomes unavailable, no new work enters the system, but in-flight documents continue processing because workers already hold queue leases and page artifacts remain in object storage. At each step, data is checkpointed and the processing status is updated in the database, ensuring that if a step fails, processing can resume from the point of failure with checkpointed data reloaded into mem-

ory. If workers restart, the queue redelivery mechanism provides recovery without requiring model state to be reconstructed inside the orchestration tier. If the Inference Service is unavailable or saturated, workers block or retry at a well-defined service boundary rather than failing the entire control plane. In practice, this isolation makes operational debugging substantially easier because failures can be localized to ingress, orchestration, inference, or downstream systems instead of appearing as a single monolithic outage.

3.6 When This Architecture Is Appropriate

This architecture is most useful when document volume, page count, or model cost is high enough that per-step scaling and cost control matter. It is also useful when teams need control over model selection, data residency, intermediate artifacts, or human-review workflows. For low-volume, homogeneous forms, a managed document-AI platform or a simpler monolithic worker may be cheaper to operate.

We considered four common alternatives. A managed intelligent-document-processing platform can reduce engineering effort, but bespoke, dataset-specific pipelines typically achieve substantially higher extraction accuracy on the document types that matter to a given deployment—managed platforms also limit model choice, observability into intermediate artifacts, and deployment control. A monolithic worker that performs ingestion, OCR, parsing, and result delivery is simpler, but couples CPU orchestration to GPU or API inference capacity. Embedding OCR models directly inside every worker removes a network call, but its co-located resource profile is poor in both directions: GPU-backed workers idle their accelerators during the majority of each task that is spent on I/O and orchestration (DocTR alone reserves ~ 800 MB of GPU memory per worker, §3.3), while CPU-only workers run OCR 3–5 $\times$ slower and bottleneck the pipeline. A VLM-only pipeline is attractive because it bypasses OCR and text stitching, but it usually increases per-page cost, complicates auditability, and makes it harder to preserve word-level evidence. The proposed design accepts more infrastructure complexity in exchange for independent scaling, model replaceability, and clearer operational boundaries.

4 Pipeline Design

Each document flows through a configurable sequence of pipeline steps, defined per document type in a YAML configuration. The pipeline implements a modular extract-transform-load pattern where each step receives the accumulated context from prior steps and appends its results.

Table 1: Classification strategy comparison. The hybrid approach achieves near-VLM accuracy at near-CLIP cost by using VLM as a selective fallback (4% of pages).

Strategy	Accuracy	Latency/page	Cost/page
CLIP-KNN	92%	0.5–1 s	\$0.000
VLM	98%	2–3 s	\$0.010
Hybrid	96%	0.6–1.2 s	\$0.001

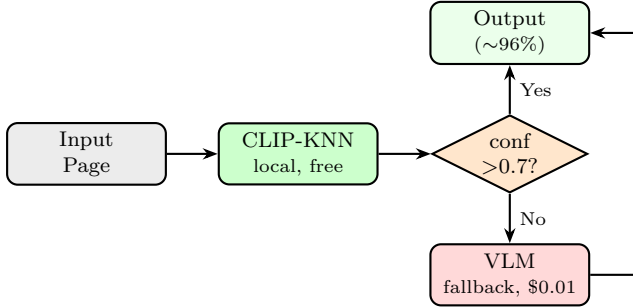


Figure 2: Hybrid classification strategy. CLIP-KNN classifies each page locally with no API cost. When confidence exceeds 0.7 (~96% of pages), the prediction is accepted directly. Low-confidence pages fall back to a VLM classifier at higher latency and cost.

4.1 Step 1: Classification

Classification determines the type for each page in a multi-page submission. This routing decision determines which extraction pipeline and schema (for structured outputs) apply downstream.

We implement a **hybrid classification strategy** that balances cost, latency, and accuracy (Table 1). The primary classifier uses CLIP embeddings [Radford et al., 2021] with a k-nearest-neighbor (KNN) index trained on representative page images. CLIP-KNN runs locally with no API cost at 0.5–1 s per page, achieving 92% accuracy. When CLIP-KNN confidence falls below a threshold, the system falls back to a VLM classifier that sends the page image to Anthropic’s Claude (Sonnet family) [Anthropic, 2024] for classification.

In our tests so far, the hybrid strategy triggers VLM fallback on only 4% of pages, reducing direct model/API cost by roughly 10× compared to VLM-only classification while recovering most of the accuracy gap. These figures exclude cluster infrastructure, storage, observability, engineering operations, and human-review labor. Figure 2 illustrates the decision flow. The CLIP-KNN index is stored locally and updated through MLflow model registry, enabling retraining without code changes.

4.2 Step 2: Auxiliary Metadata Extraction

For submissions requiring timeliness or provenance verification, a dedicated step extracts auxiliary metadata

(e.g., date stamps, barcodes, signatures) from cover and supplementary pages. We support two backends: an RF-DETR [Robinson et al., 2025] object detection model that localizes target regions followed by recognition, and a Claude Sonnet-based extractor that processes the full page image. The object detection approach is faster but requires training data for each target format; the Claude Sonnet approach generalizes better to novel formats.

4.3 Step 3: OCR

OCR converts page images to text with word-level bounding boxes. Our primary OCR engine is DocTR [Mindee, 2021], a PyTorch-based two-stage pipeline using a `dp_resnet50` text detection network and a `crnn_vgg16_bn` recognition network. DocTR processes pages at 1–2 s per page on GPU, producing word-level text with confidence scores and bounding box coordinates.

As we discuss in §5, OCR is the dominant bottleneck in the pipeline, consuming a large majority of end-to-end execution time for a typical multi-page document. This motivates the Inference Service’s design: by isolating OCR inference, we can scale GPU resources specifically to address this bottleneck without over-provisioning the rest of the system.

4.4 Step 4: Text Stitching

Multi-page documents require combining OCR output from individual pages into a coherent text representation. The stitching step concatenates per-page text in page order, preserving page boundaries as metadata for downstream extraction. This step is computationally trivial (<1 s) but architecturally important: it transforms the per-page OCR output into a document-level representation that the parsing step can reason over.

4.5 Step 5: Structured Parsing

The final extraction step sends the stitched OCR text to Claude Sonnet with a form-specific prompt and JSON schema. The LLM maps unstructured OCR text to structured field-value pairs, handling the semantic interpretation that rule-based extraction cannot: resolving ambiguous field references, interpreting checkbox states from OCR context, and validating internal consistency.

Each page type defines its own schema, dynamically generated from Pydantic models for type-safe validation of LLM output; the schema generator converts these to JSON Schema for inclusion in the LLM prompt.

Parsing typically consumes ~4,500 input tokens (for an 8-page document’s OCR text) and produces ~400 output tokens of structured JSON, with a latency of ~3 s and a cost of ~\$0.03 per document. Combined with hybrid classification at \$0.001 per page, the total direct API cost for an 8-page document is roughly \$0.038—approximately 80% from parsing and 20% from classification. While

cheaper than running a VLM on raw images, this parsing cost dominates the per-document budget and motivates our decision to use OCR for text extraction rather than sending all pages directly to a VLM.

5 A Case Study: Batch Processing at Scale

To characterize the system’s scaling behavior in practice, we profiled the full pipeline under controlled batch workloads of several hundred synthetic multi-page documents. We report qualitative findings here; detailed quantitative benchmarks are left to future work, pending a profiling re-run with the instrumentation improvements discussed in §7 (e.g., corrected stale-detection thresholds, explicit retry accounting, and decoupled queue-depth tuning).

5.1 Single-Document Execution Profile

For a typical multi-page document, OCR dominates end-to-end wall-clock time. A representative 8-page document spends roughly two-thirds of its processing time in OCR, with LLM-based structured parsing a distant second: parsing runs once over the stitched OCR text rather than per page, so parsing latency grows sublinearly with page count—in contrast to OCR, which scales linearly because each page is processed independently. Initialization, document creation, text stitching, result upload, and database updates collectively account for a small share of execution time. Per-page OCR latency is on the order of 1–2 s on GPU, consistent with DocTR benchmarks, and peak memory usage stays around 1 GB, with DocTR model weights accounting for ~800 MB.

5.2 Concurrency Saturation

Under increasing concurrent load, throughput improves sharply at low concurrency and then flattens once the Inference Service’s GPU capacity saturates—additional workers queue behind inference requests rather than processing in parallel. In our deployments, this inflection occurs near the “pods $\times$ tasks per pod” product that matches the Inference Service’s steady-state request rate.

P95 per-document latency—the 95th-percentile end-to-end processing time across documents, a standard tail-latency metric in production systems—remains stable across concurrency levels below saturation, a direct benefit of the message queue mediated communication: workers wait for queue messages rather than overwhelming the Inference Service with simultaneous requests. Once concurrency exceeds the GPU inference capacity, tail latency degrades as requests queue at the Inference Service, reinforcing that the Inference Service—not orchestration—sets the saturation ceiling. Figure 3 sketches this behavior qualitatively.

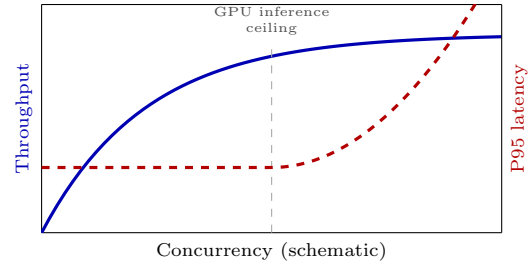


Figure 3: Schematic saturation behavior. Throughput (solid, blue) rises at low concurrency, then flattens once the Inference Service’s GPU capacity saturates. P95 per-document latency (dashed, red) stays approximately flat below the ceiling and degrades as requests queue at the Inference Service above it. Axes are illustrative; actual saturation thresholds depend on workload, model, and pod sizing.

5.3 Multi-Pod Scaling

Scaling worker pods beyond the concurrency required to saturate a single Inference Service pod yields diminishing returns; the bottleneck shifts from orchestration to GPU inference. Adding Inference Service replicas unlocks further scaling, but with its own diminishing returns as the next-tier bottleneck—object-storage I/O, LLM API rate limits, or queue overhead—begins to dominate. Figure 4 summarizes this tiered progression. The key architectural takeaways—that inference and orchestration must scale independently and that the Inference Service is typically the first tier to saturate—are robust across configurations. The specific break-even ratio between worker and Inference Service pods is workload-dependent and requires profiling with corrected instrumentation to state quantitatively.

One solution to the inference bottleneck might be to further decouple the inference service into services of independent operations, e.g. a classifier service, an OCR service, etc. However, we found that this approach reduced the benefits of staggering and made scaling decisions more complex, while also requiring additional engineering and devops effort. As inference is the primary cost driver for a document processing architecture, this resource is generally externally constrained and therefore a simple, horizontally scaling solution is generally the best approach.

6 Infrastructure for Model Integration

A production document understanding system must accommodate model evolution and improvements without system redesign. The architecture addresses this through three mechanisms.

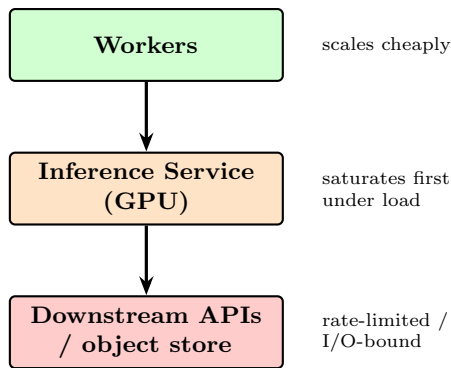


Figure 4: Tiered bottleneck progression. Worker pods scale horizontally at low cost; the Inference Service’s GPU capacity is the first tier to saturate as concurrency grows; adding Inference Service replicas then shifts the bottleneck to downstream services (LLM APIs, object storage) and queue overhead.

6.1 Containerized Inference

Each OCR model is packaged as a Docker container exposing a standard REST interface. The Inference Service loads the configured model at startup and serves predictions via FastAPI endpoints. Swapping models requires changing a configuration value and redeploying the Inference Service—worker code remains untouched because it communicates through a stable API contract.

We have deployed DocTR (PyTorch, GPU), Docling (CPU-optimized), and SmolDocling (vision transformer, GPU) through this mechanism. The consistent interface allows A/B testing across models by routing a fraction of traffic to an alternate Inference Service deployment.

6.2 Model Registry

Models are versioned and promoted through an MLflow model registry with environment-based aliases (development, staging, production). The CLIP-KNN classification index, RF-DETR detection weights, and OCR models all flow through this registry, providing reproducible deployments and rollback capability. Each model artifact is tagged with training metadata, evaluation metrics, and the dataset version used for training.

6.3 Local vs. API Inference

The system supports both local model inference (DocTR, CLIP, RF-DETR running on cluster GPUs) and cloud API inference (Claude Sonnet via the Anthropic API). The choice is configured per pipeline step:

- **Classification:** Local CLIP-KNN with Claude Sonnet fallback.
- **Postmark detection** (for example on envelopes for scanned correspondence): Local RF-DETR or Claude Sonnet (configurable).
- **OCR:** Local DocTR (GPU) or Docling (CPU).

- **Parsing:** Currently API-only (Claude Sonnet via Anthropic API).

This hybrid approach minimizes API costs for high-volume steps (classification: \$0.001/page via mostly-local inference) while leveraging cloud APIs for steps where model capability matters more than cost (parsing: \$0.03/document but only one call per document). Sensitive deployments require additional controls around provider data retention, private networking, encryption, audit logging, and retention policies; in stricter environments, the same service contract can be backed by self-hosted models or private model endpoints rather than public APIs.

6.4 Observability

Production operation depends on observability at the document, step, and service levels. At minimum, the system must expose document-level status transitions, per-step latency, queue depth, retry counts, and failure attribution across the Gateway, Worker, and Inference Service tiers. These signals serve different purposes: queue depth and worker concurrency reveal whether orchestration is keeping pace with ingress; inference latency and GPU saturation reveal when model-serving capacity is the active bottleneck; document-level statuses and structured error codes reveal whether failures are concentrated in OCR, parsing, object-storage access, or downstream API dependencies.

This distinction matters because many symptoms look similar from the outside. A growing queue might indicate underprovisioned workers, an inference bottleneck, or repeated retries caused by an unreliable downstream dependency. Likewise, a rise in end-to-end document latency might reflect OCR slowdown, API throttling, or queue contention rather than a regression in the final parsing step. The architecture therefore benefits from observability aligned to service boundaries and pipeline steps, not just coarse application-level success rates.

7 Lessons Learned

Operating this architecture in production revealed several failure modes and design insights not apparent from model benchmarks alone.

Message queue visibility timeout must match processing time. Our initial visibility timeout of 30s caused documents to be re-delivered to other workers while still being processed (typical processing takes 15–25s per document). This produced duplicate results and wasted compute. Setting the visibility timeout to 300s—well above the P99 processing time—eliminated re-delivery without meaningfully delaying failure detection.

OCR is the bottleneck, not the LLM. Intuition suggested that LLM parsing would dominate latency and cost. In practice, OCR dominates end-to-end execution time while LLM parsing takes a much smaller share. This is because OCR processes every page independently (e.g., 8 sequential inference calls for an 8-page document), while parsing processes the full document in a single LLM call. Optimizing OCR throughput—through batched inference, model distillation, or faster architectures—yields greater system-level improvements than optimizing the LLM step.

Model accuracy $\neq$ system reliability. A VLM classifier achieving 98% accuracy in evaluation still produces 2% misclassifications in production, routing documents to incorrect extraction pipelines. At 1,000 documents/day, this means 20 daily failures requiring human review. The hybrid classification strategy mitigates this challenge by using CLIP-KNN’s different failure distribution as a complementary signal, but the lesson generalizes: system reliability requires defense in depth, not just high model accuracy.

Stale detection is harder than it sounds. Workers must detect when a document has been processing for too long and is likely stuck. Our initial implementation tracked time from when the document entered the worker’s local queue, not from when inference actually started. Documents waiting in the local queue for inference capacity appeared “stale” and were incorrectly marked as failed. The fix—tracking `processing_start_time` from the first Inference Service call—required careful state management across async tasks.

Changing models requires scaling reanalysis. While staggering is a useful technique for us to improve resource utilization, it also required us to consider the implications of adding new models to the pipeline. Generally our experimentation focused on improving accuracy or model performance, but we found that we needed to additionally consider the impact on the worker-to-inference ratio and the potential for resource contention. Generally speaking, given two models that are approximately equivalent in accuracy, the model that uses different computational resources than the previous or next step in the pipeline is preferable even if the throughput is lower on a per-document basis.

Limitations of the present case study. The empirical discussion in this paper is intentionally scoped as an operational case study rather than a fully controlled systems benchmark. Saturation points, worker-to-inference ratios, and per-step cost tradeoffs depend on workload composition, page count distribution, document quality, queue settings, model choice, and the behavior of downstream managed APIs. Our observations therefore support the architectural claims qualitatively and directionally, but

they should not be treated as universal constants. Teams adopting similar designs should re-profile with their own workloads, hardware, and retry policies before transferring the exact thresholds reported here.

8 Conclusion

We have described a production architecture for document understanding that processes structured forms through a queue-driven pipeline of classification, OCR, and language-model-based extraction. The system’s key architectural decisions—separating inference from orchestration, using hybrid classification to balance cost and accuracy, and communicating through message queues for decoupled scaling—reflect tradeoffs that we believe generalize beyond our specific deployment.

The qualitative profiling observations presented here are intended to help practitioners make informed infrastructure decisions before investing in a full benchmark suite. Our finding that OCR, not LLM parsing, dominates end-to-end latency may surprise teams planning document understanding deployments, and motivates investment in OCR optimization or alternative architectures that bypass OCR entirely.

Looking ahead, we see three directions for further evolution of this class of systems. First, end-to-end VLMs that can process page images directly may eventually replace the OCR step, simplifying the pipeline at the cost of higher per-page inference cost. Second, retrieval-augmented approaches [Faysse et al., 2024] could enable selective processing of only relevant pages from large document collections, reducing total compute. Third, efficiency-optimized models may shift the cost-accuracy Pareto frontier enough to make VLM-only pipelines economically viable at scale.

The gap between model research and production deployment remains wide. We hope that describing this architecture, its operating characteristics, and the lessons we learned running it contributes to closing that gap.

References

- Anthropic. The Claude model family. <https://claude.com/product/overview>, 2024.
- Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Berrospi Ramis, Matteo Omenetti, Fabian Lindlbauer, Kasper Dinkla, Lokesh Mishra, Yusik Kim, Shubham Gupta, Rafael Teixeira de Lima, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk, and Peter W. J. Staar. Docling technical report. *arXiv preprint arXiv:2408.09869*, 2024.
- Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-VL: A versatile vision-language model

- for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.
- Denis Baylor, Eric Breck, Heng-Tze Cheng, Noah Fiedel, Chuan Yu Foo, Zakaria Haque, Salem Haykal, Mustafa Ispir, Vihan Jain, Levent Koc, et al. TFX: A TensorFlow-based production-scale machine learning platform. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1387–1395, 2017.
- Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. Nougat: Neural optical understanding for academic documents. *arXiv preprint arXiv:2308.13418*, 2023.
- Lilu Cheng, Jingjun Lu, Yi Xuan Chan, Quoc Khai Nguyen, John Bi, and Sean Ho. A hybrid architecture for multi-stage claim document understanding: Combining vision-language models and machine learning for real-time processing. *arXiv preprint arXiv:2601.01897*, 2026.
- Yuning Du, Chenxia Li, Ruoyu Guo, Xiaoting Yin, Weiwei Liu, Jun Zhou, Yifan Bai, Zilin Yu, Yehua Yang, Qingqing Dang, and Haoshuang Wang. PP-OCR: A practical ultra lightweight OCR system. *arXiv preprint arXiv:2009.09941*, 2020.
- Jeffrey Dunn. Introducing FBLeaRner Flow: Facebook’s AI backbone. Meta Engineering Blog, <https://engineering.fb.com/2016/05/09/core-infra/introducing-fblearner-flow-facebook-s-ai-backbone/>, 2016.
- Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. ColPali: Efficient document retrieval with vision language models. *arXiv preprint arXiv:2407.01449*, 2024.
- Yao Fehlis, Zhangzhang Si, Steve Kramer, Analysa Gonzales, and Michael Wharton. Practical guide on document understanding: From OCR to VLM. <https://doi.org/10.5281/zenodo.18020024>, dec 2025.
- Google DeepMind. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.
- Jeremy Hermann and Mike Del Balso. Meet Michelangelo: Uber’s machine learning platform. Uber Engineering Blog, <https://www.uber.com/blog/michelangelo-machine-learning-platform/>, 2017.
- Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia*, pages 4083–4091, 2022.
- Md Mofijul Islam, Md Sirajus Salekin, Joe King, Priyashree Roy, Vamsi Thilak Gudi, Spencer Romo, Akhil Nooney, David Kaleko, Boyi Xie, Bob Strahan, and Diego A. Socolinsky. IDP accelerator: Agentic document intelligence from extraction to compliance validation. *arXiv preprint arXiv:2602.23481*, 2026.
- Guillaume Jaume, Hazim Kemal Ekenel, and Jean-Philippe Thiran. FUNSD: A dataset for form understanding in noisy scanned documents. In *International Conference on Document Analysis and Recognition Workshops*, pages 1–6, 2019.
- Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. OCR-free document understanding transformer. In *European Conference on Computer Vision*, pages 498–517. Springer, 2022.
- Kenton Lee, Mandar Joshi, Iulia Turc, Hexiang Hu, Fangyu Liu, Julian Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2Struct: Screenshot parsing as pretraining for visual language understanding. *Proceedings of the 40th International Conference on Machine Learning*, pages 18893–18912, 2023.
- Minesh Mathew, Dimosthenis Karatzas, and C.V. Jawahar. DocVQA: A dataset for VQA on document images. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 2200–2209, 2021.
- Mindee. docTR: Document text recognition. <https://github.com/mindee/doctr>, 2021.
- Mor Shpigel Nacson, Aviad Aberdam, Roy Ganz, Elad Ben Avraham, Alona Golts, Yair Kittenplon, Shai Mazor, and Ron Litman. DocVLM: Make your VLM an efficient reader. *arXiv preprint arXiv:2412.08746*, 2024.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763, 2021.
- Ricoh. fi-8950 Production Scanner. Product Datasheet, 2024. URL <https://www.ricoh-usa.com/en/products/pd/equipment/scanners/fi-8950-production-scanner>. Accessed: 2026-05-11.
- Isaac Robinson, Peter Robicheaux, Matvei Popov, Deva Ramanan, and Neehar Peri. RF-DETR: Neural architecture search for real-time detection transformers. *arXiv preprint arXiv:2511.09554*, 2025.

Alexandre Sallinen, Stefan Krsteski, Paul Teiletche, Marc-Antoine Allard, Baptiste Lecoeur, Michael Zhang, Fabrice Nemo, David Kalajdzic, Matthias Meyer, and Mary-Anne Hartley. MMORE: Massive multimodal open RAG & extraction. *arXiv preprint arXiv:2509.11937*, 2025.

Jiyuan Shen, Peiyue Yuan, Atin Ghosh, Yifan Mai, and Daniel Dahlmeier. OCR or not? rethinking document information extraction in the MLLMs era with real-world large-scale datasets. *arXiv preprint arXiv:2603.02789*, 2026.

Ray Smith. An overview of the Tesseract OCR engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, pages 629–633, 2007.

Zilong Wang and Xiaoyu Shen. Hybrid OCR-LLM framework for enterprise-scale document information extraction under copy-heavy task. *arXiv preprint arXiv:2510.10138*, 2025.

Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. LayoutLM: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1192–1200, 2020.